

Project Part B: Strategy Report

Team RNgang

1. Approach

1.1 Search Algorithm

We implemented minimax search with alpha-beta pruning to a depth of 3. The algorithm searches from our agent's perspective (maximising score), modelling the opponent as a minimising player. Alpha-beta pruning eliminates branches where the current path cannot improve on an already-found bound, reducing the tree size without affecting the selected action.

Each stack generates at most 12 candidate actions (3 types x 4 directions), so with n stacks the branching factor is at most $b = 12n$, so in the early phase with 4 stacks per side, $b \leq 48$. Worst-case node count at depth 3 becomes $O(b^3)$. Alpha-beta pruning significantly reduces the number of explored nodes by pruning branches that cannot affect the final decision. The best-case scenario using alpha-beta pruning with perfect move ordering reduces this to $O(b^{3/2})$. In practice, move ordering (Section 1.2) achieves close to best-case reduction, keeping per-turn search within the 180-second CPU budget set. As stacks merge or are eliminated, n decreases, reducing the branching factor and making late-game search cheaper.

We selected a depth of 3 after timing a depth of 4 implementation on a mid-game board (8 stacks). Depth 3 required approximately 0.23s CPU per turn while depth 4 required approximately 2.3s, showing a 10 times increase. At 300 play-phase turns, depth 4 would consume around 690s in the worst case, exceeding the 180s CPU budget, while depth 3 consumes approximately 68s over a full game, leaving a suitable margin.

Terminal states stop evaluation at any depth, and positive infinity is returned immediately if the opponent has no tokens, negative infinity if we do. If a position has no legal actions, 0 is returned as a stalemate draw.

1.2 Move Ordering

Before expanding children at any node, actions are sorted by EAT, CASCADE, then MOVE. EAT actions directly reduce the opponent's token count and usually results in high-scoring positions, so evaluating them first tightens alpha and beta bounds early and prunes more of the remaining subtree. CASCADE is second as it could push stacks off the board and significantly reorganise positions. MOVE is tried last as the least effective.

1.3 Evaluation Function

Non-terminal board states are scored using a linear weighted sum of three features:

$$\text{EVAL}(s) = W_{\text{TOKENS}} \times \text{token_diff} + W_{\text{HEIGHTS}} \times \text{height_diff} + W_{\text{THREATS}} \times \text{threat_diff}$$

Feature	Weight	Computation	Strategic Motivation
token_diff	10	sum(my heights) - sum(opp heights)	As the win condition, tokens are the primary metric
height_diff	3	max(my heights) - max(opp heights)	The taller the stack, the more powerful (CASCADE or EAT)
threat_diff	2	my eat threats - opp eat threats	Penalises positions where the opponent can EAT our stacks

Table 1.1: Evaluation function features, weights, and strategic motivations.

Before performing weight tuning evaluations (Section 2.1) to obtain appropriate weights, we created this evaluation function. `token_diff` carries the highest weight since eliminating all opponent tokens is the win condition. `height_diff` uses the maximum stack height, since a single tall stack can have a stronger impact on available moves and outcomes. A stack of height h can cascade h cells and eat any adjacent stack of height $\leq h$. Therefore rewarding max height encourages building one powerful stack. Since total height is already indirectly captured by `token_diff`, using the maximum instead of the sum avoids redundancy while capturing more information. `threat_diff` counts the number of stacks where an opponent's EAT move is possible. Positive or zero values indicate favourable or neutral defensive positions where we are not at immediate risk of losing tokens. Negative value signals that we may lose tokens, so we want to avoid these positions.

1.4 Placement Strategy

During the placement phase (4 turns each player), each valid cell is scored as:

$$\text{score} = -2 \times \text{center_dist} + 0.5 \times \text{spread}$$

`center_dist` is the Manhattan distance from the cell to the board centre (3.5, 3.5). `spread` is the minimum Manhattan distance from the candidate to any existing friendly stack. Cells closer to the centre are preferred because they provide more movement options (all 4 directions) from the start of the game. The spread term discourages clustering, distributing stacks across the board maximises coverage when entering the play phase.

2. Performance Evaluation

2.1 Evaluation Weight Tuning

To select the evaluation weights, we wrote a tuning script that tested weight combinations against a tokens-only baseline ($W_TOKENS=10$, $W_HEIGHTS=0$, $W_THREATS=0$) over 10 games per combination (5 as each colour). W_TOKENS was fixed at 10 throughout. The 3x3 win rate grid below shows results across $W_HEIGHTS$ in $\{0, 3, 6\}$ and $W_THREATS$ in $\{0, 2, 5\}$:

		W_THREATS		
		0	2	5
W_HEIGHTS	0	0.50	1.00	0.50
	3	1.00	1.00	1.00
	6	0.50	1.00	1.00

Table 2.1: Win rate vs tokens-only baseline

From the table, we can see that neither feature alone at extreme values reliably wins: $W_HEIGHTS=6$ with $W_THREATS=0$ draws with the baseline, as does $W_HEIGHTS=0$ with $W_THREATS=5$. Overweighting a single feature causes the agent to neglect the other dimension entirely. On the contrary, moderate combinations of both features consistently beat the baseline. $W_HEIGHTS=3$ paired with any $W_THREATS$ value wins 100%, as does $W_THREATS=2$ paired with any $W_HEIGHTS$ value. Our chosen weights ($W_TOKENS=10$, $W_HEIGHTS=3$, $W_THREATS=2$) fall in this reliable region and are further supported by the strategic reasoning in Section 1.3.

Since multiple combinations achieved 100% against the baseline, we ran head-to-head games between the 100% performers and chosen configuration (10, 3, 2):

Opponent (W_TOKENS , $W_HEIGHTS$, $W_THREATS$)	Wins	Losses	Draws
(10, 0, 5)	1	1	0
(10, 3, 0)	0	0	2
(10, 3, 5)	0	0	2
(10, 6, 0)	2	0	0
(10, 6, 2)	2	0	0
(10, 6, 5)	1	0	1

Table 2.2: (10, 3, 2) head-to-head vs other 100% performers

The key finding is that $W_HEIGHTS = 6$ is consistently beaten by $W_HEIGHTS = 3$. This means that overweighting stack height leads the agent to pursue aggressive stacking at the cost of total token count, which is what determines the winner. $W_HEIGHTS = 3$ strikes the right balance as it rewards building a strong stack without neglecting token parity.

The threat weight ($W_THREATS = 2$) draws consistently against $W_THREATS = 0$ and $W_THREATS = 5$ at $W_HEIGHTS = 3$, suggesting that within this region the exact threat weight is less critical. We retain $W_THREATS = 2$ as a small contribution to defensive awareness. Our final weights are confirmed as ($W_TOKENS=10$, $W_HEIGHTS=3$, $W_THREATS=2$).

2.2 Iterative Deepening

We implemented an iterative deepening variant of alpha-beta that searches depth 1, 2, 3, ... until a per-move CPU budget of 0.5s is exhausted, then returns the best action from the deepest completed depth. We tried this because fixed-depth search can be inefficient in late-game states where the number of stacks is reduced, leading to a lower branching factor. In these positions, the same time budget may allow deeper search than in the early game, potentially improving decision quality.

In practice, this implementation reached depth 3 on every turn. The reduced branching factor in late-game positions was insufficient to reliably complete depth 4 within the time budget. Additionally, by the time depth 4 became feasible, many positions were already close to terminal states, limiting the benefit of deeper search. Over 10 head-to-head games against the fixed-depth agent, this implementation produced 10 draws, suggesting equivalent performance. Therefore, we retained using a fixed depth 3 as it provides identical performance with lower computational cost.

3. Other Aspects

3.1 Board Representation

The board is stored as a sparse dict $\{\text{Coord: CellState}\}$, only occupied cells are present, empty cells are absent from the dictionary. Dictionary lookup and insertion are $O(1)$ average, and copying the board per search node takes $O(k)$ where k is the number of occupied cells (at most 8 after placement, falling as stacks are eliminated). This keeps per-node overhead low throughout the search tree.

3.2 Terminal Detection Depth Independence

Terminal state checks run before the depth cutoff, if either player has zero tokens, the result is returned immediately regardless of depth. This ensures wins and losses deep in the trees are correctly identified and propagated, not mistakenly treated as non-terminal.

3.3 Recursive Cascade Push Chain

The CASCADE action requires chain-pushing, where a token landing on an occupied cell pushes that stack one cell further, which may itself displace another stack, and so on until a stack either finds an empty cell or is pushed off the board. We implement this as a recursive function that first pushes the occupant of the destination before moving the source stack, this correctly handles arbitrarily long push chains as well as elimination of stacks that are pushed off the board

4. Supporting Work

A benchmarking script that simulates full games directly between two agents without using the referee, enabling faster head-to-head testing. It reports per-matchup win/loss/draw totals and average game time. All comparison data in Section 2 was generated using this script.

A weight tuning script that tests evaluation weight combinations. For each (W_HEIGHTS, W_THREATS) pair, it creates an agent variant over a parameterised evaluation function and runs it against the tokens-only baseline. Results are printed as a win-rate table, and the output directly resulted in our final weight selection.

A standalone iterative deepening alpha-beta agent was implemented for comparison with our submission implementation with a fixed depth of 3 (Section 2.2). It uses the same evaluation function and move ordering as the main agent but searches to increasing depths within a 0.5s per-move CPU budget.